

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Concept Mapping (Medium)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand	5
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand	5
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand	5
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand	5
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand	5
6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand	5

7	Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.	BL2 – Understand	5
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand	5
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand	5
10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember	5

Code of Conduct:

I Niharika [192511221] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

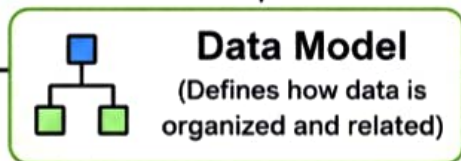
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure

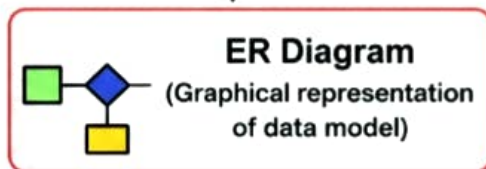
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand



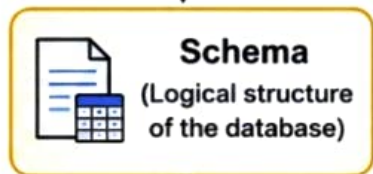
is based on



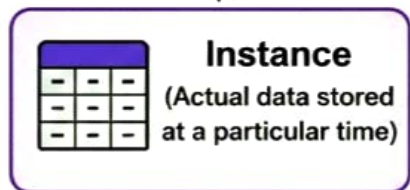
is represented using



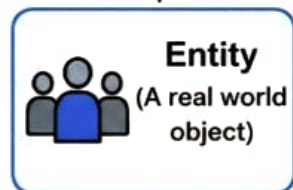
defines



describes the structure of the database



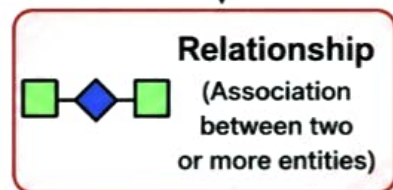
represents the data at a particular time



represents a real world object



describes the properties of an entity



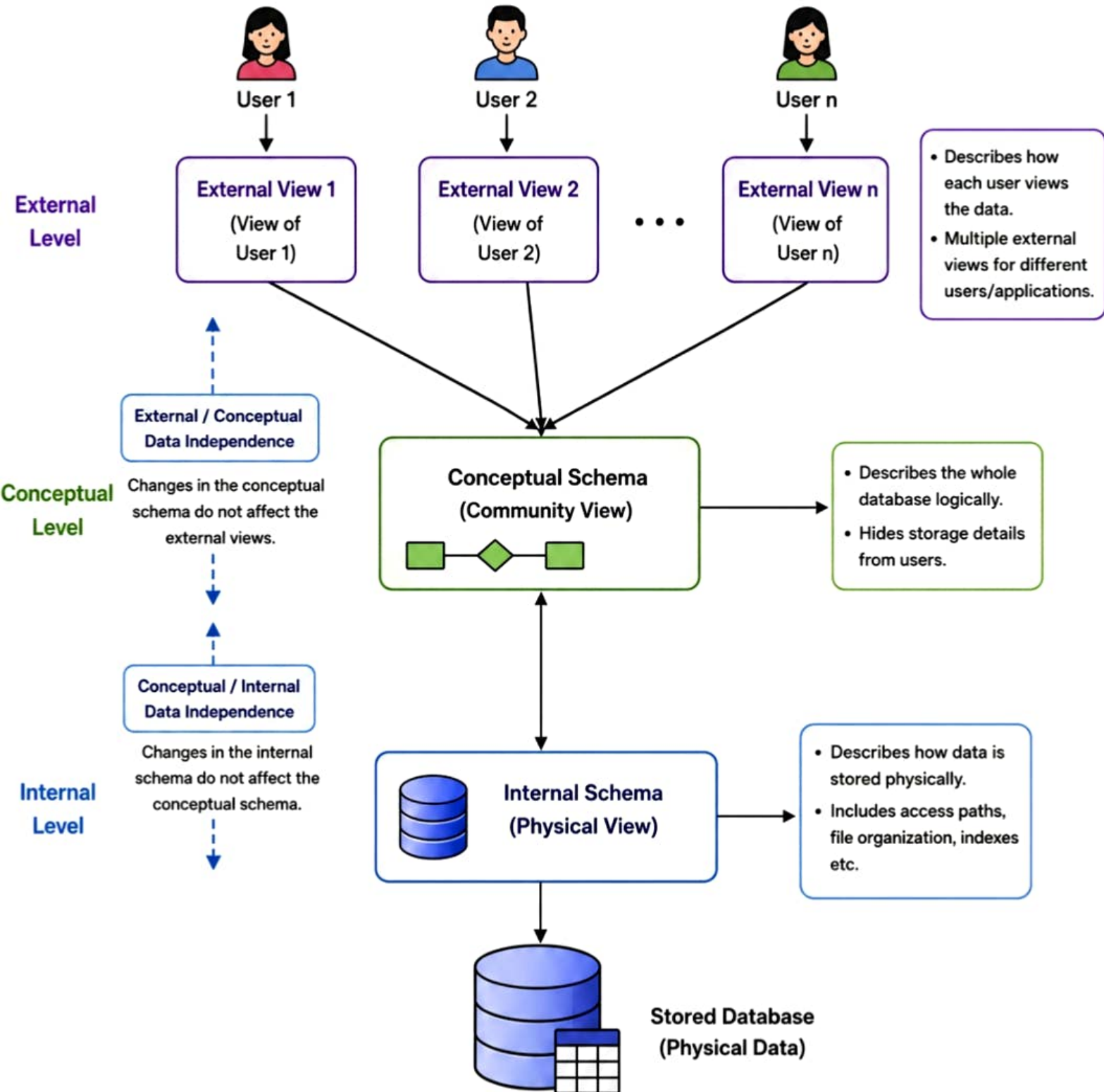
represents an association between entities

consists of

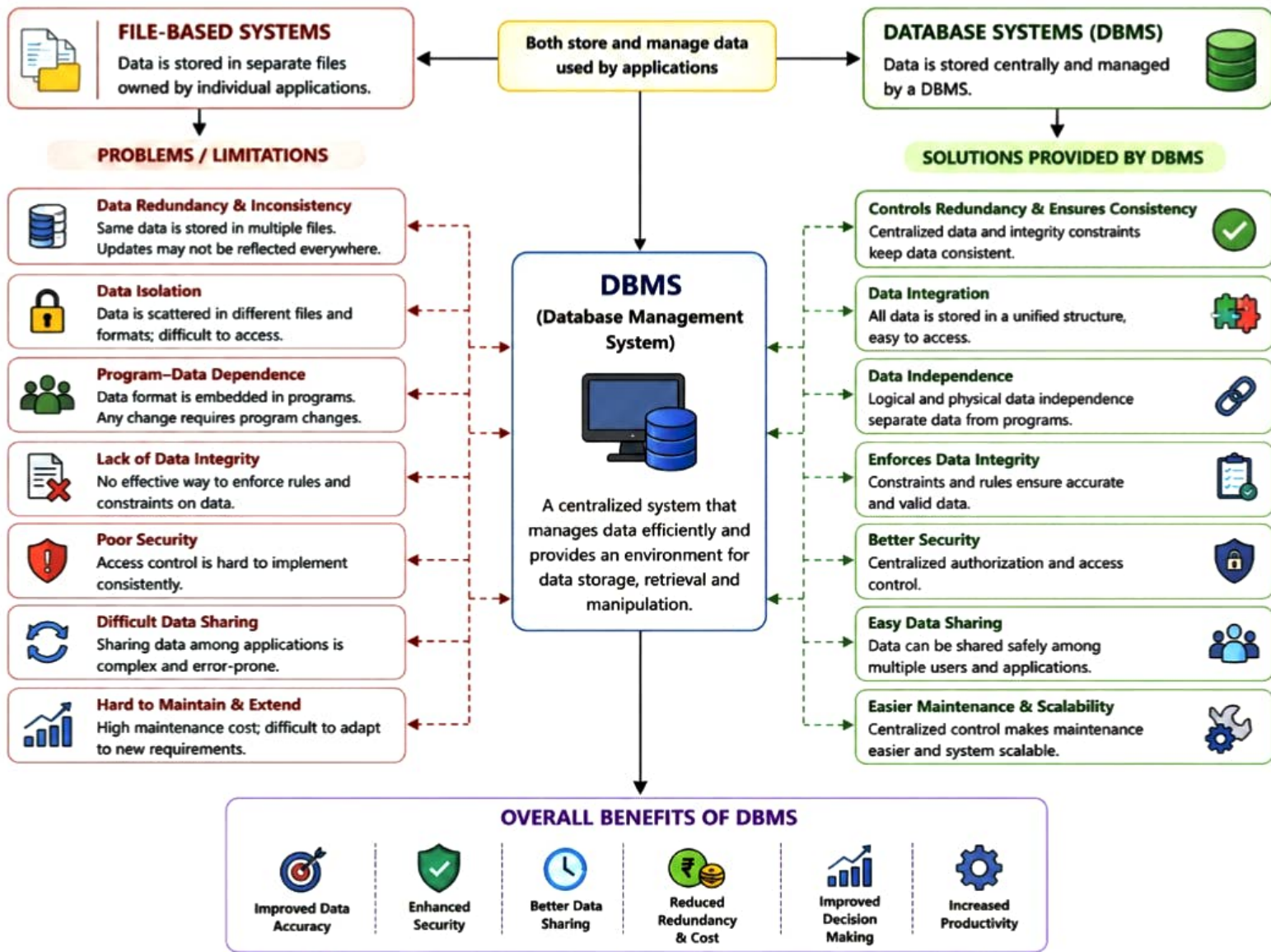
Flow Summary : DBMS → Data Model → (Schema → Instance)

and Data Model → ER Diagram → (Entity, Attribute, Relationship)

ANSI/SPARC Three-Level Architecture



File-Based Systems vs Database Systems



EER MODEL EXTENSIONS

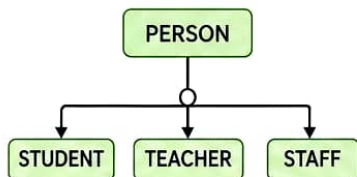
EER Model (Enhanced ER Model)

extends ER model with

GENERALIZATION

Combines multiple entity types into a supertype.

Top-Down Approach



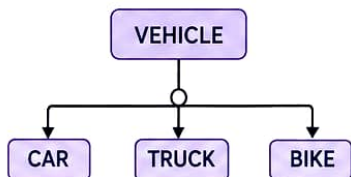
Example:

Person is a supertype of Student, Teacher, and Staff.

SPECIALIZATION

Divides a supertype into multiple subtype.

Bottom-Up Approach



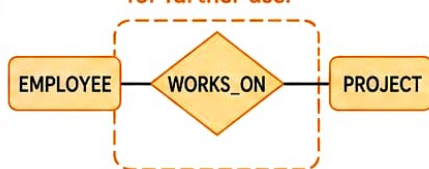
Example:

Vehicle is specialized into Car, Truck, and Bike.

AGGREGATION

Treats a relationship set as a higher-level entity.

Abstracts a relationship for further use.



Example:

WORKS_ON relationship is treated as an entity for further relationships.



These extensions improve the modeling power of ER model to represent real-world scenarios more effectively.

ER DIAGRAM COMPONENTS

(Concept Map)



1. ENTITY

An entity is a distinguishable object or concept in the real world about which data is stored.

Notation



Rectangle

Example

STUDENT

Key Points

- Has an independent existence.
- Represented by a rectangle.
- Each entity has properties (attributes).

2. WEAK ENTITY

A weak entity does not have a primary key of its own and depends on another entity (Owner entity) for its identity.

Notation



Double Rectangle

Example

DEPENDENT
(weak entity)

Key Points

- Cannot exist without its owner entity.
- Identified using partial key (discriminator).
- Represented by double rectangle.

3. ATTRIBUTE TYPES

Attributes describe the properties or characteristics of an entity or relationship.

Types of Attributes

1. Simple Attribute

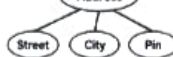
Atomic, indivisible.

Age

2. Composite Attribute

Can be divided into subparts.

Address



3. Single-valued Attribute

Has one value for an entity.

Email

4. Multi-valued Attribute

Has multiple values for an entity.

Phone_No

5. Derived Attribute

Can be derived from other attribute(s).

Age

(from DOB)

Key Point

Attributes are represented by ovals.
Different types are shown using different notations.

4. RELATIONSHIP TYPES

A relationship is an association among two or more entities.

Notation



Diamond

Types

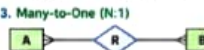
1. One-to-One (1:1)

(Each A is related to one B and vice versa)



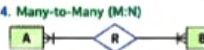
2. One-to-Many (1:N)

(One A is related to many B's)



3. Many-to-One (N:1)

(Many A's are related to one B)



4. Many-to-Many (M:N)

(Many A's are related to many B's)



Key Point

Relationships are represented by diamonds and have cardinality (1:1, 1:N, N:1, M:N).

5. PARTICIPATION CONSTRAINTS

It specifies whether the participation of an entity in a relationship is mandatory or optional.

Types

1. Total Participation (Mandatory)

Every entity must participate in the relationship.



2. Partial Participation (Optional)

Some entities may or may not participate.



Key Point

Total participation is shown by double line; partial participation by single line.

LEGEND



Entity



Weak Entity



Attribute



Multi-valued Attribute



Derived Attribute



Relationship



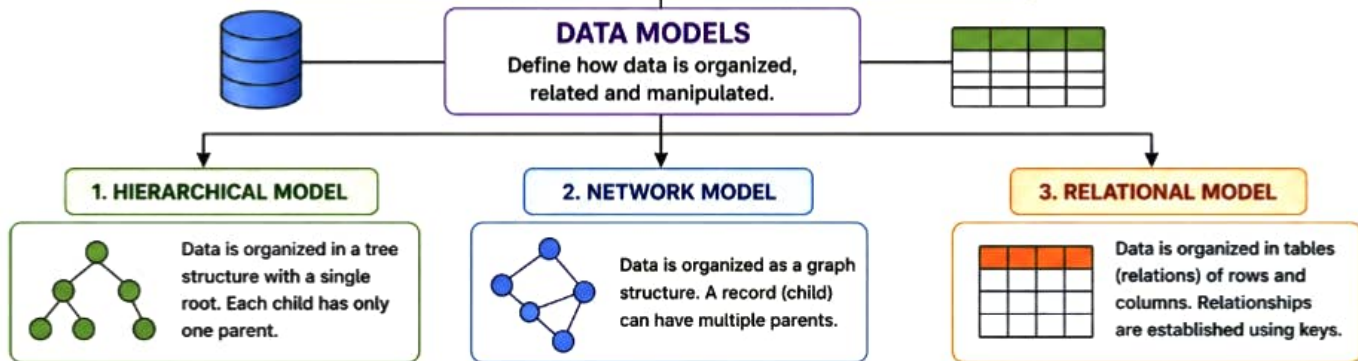
Partial Participation



Total Participation

COMPARISON OF DATA MODELS

(Concept Map)



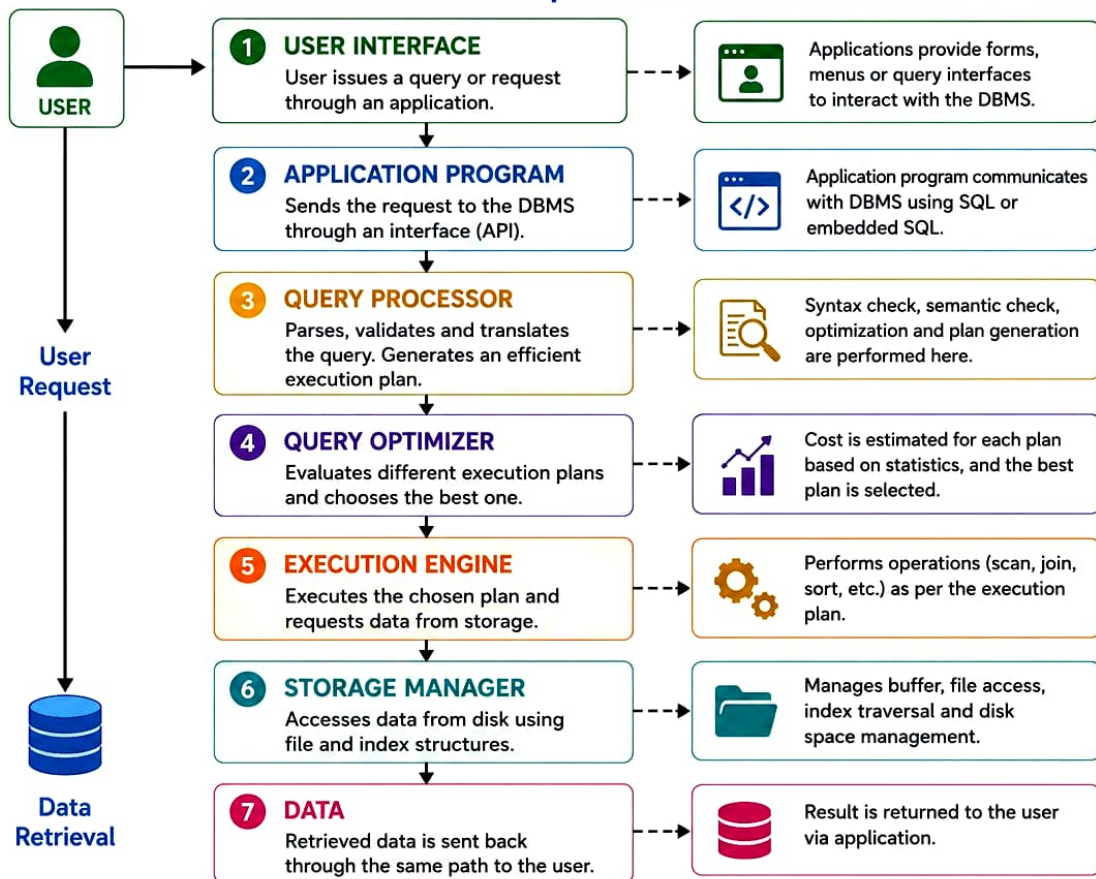
 STRUCTURE	Tree structure (one-to-many) 	Graph structure (many-to-many) 	Table structure (relations) 
 RELATIONSHIP	Each child has only one parent.	A child can have multiple parents.	Relationships via primary and foreign keys.
 DATA ACCESS	Navigation based (top-down traversal from root).	Navigation based using sets and links.	Non-navigational; uses SQL for high-level access.
 INTEGRITY	Limited integrity constraints.	More integrity than hierarchical but complex.	Strong integrity constraints (keys, domains, referential integrity).
 FLEXIBILITY	Rigid structure; hard to modify.	More flexible than hierarchical but complex to design.	Highly flexible; easy to modify and maintain.
 SCALABILITY	Not suitable for large and complex data.	Better than hierarchical; can handle complex data.	Highly scalable; suitable for large databases.
 EXAMPLES	IBM Information Management System (IMS)	CODASYL DBTG, IDMS	MySQL, Oracle, SQL Server, PostgreSQL, DB2

KEY TAKEAWAY

Hierarchical = Simplicity (one parent) • **Network** = Power (many parents) • **Relational** = Simplicity + Flexibility + Integrity

DBMS SOFTWARE ARCHITECTURE

Flow from User Request to Data Retrieval



ARCHITECTURE SUMMARY

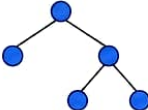
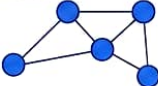
- DBMS acts as an interface between users/applications and the physical database.
- Each component has a specific role in processing the request efficiently.
- The architecture ensures data independence, security, concurrency control and efficient retrieval.

KEY BENEFITS

-  **Security & Integrity**
Ensures data security and integrity.
-  **Efficiency**
Optimized query processing for faster results.
-  **Data Independence**
Logical and physical data independence.
-  **Concurrency Control**
Handles multiple users accessing data simultaneously.

DATA MODELS AND REAL-WORLD APPLICATIONS

Mapping data models to practical use and their suitability

DATA MODEL	DESCRIPTION	REAL-WORLD APPLICATIONS	SUITABILITY	KEY FEATURES
1. HIERARCHICAL MODEL 	- Data is organized in a tree-like structure. - Each child has only one parent.	 File systems (e.g., Windows Explorer)  Organization charts  XML data handling	 Suitable for data with strict parent-child relationships.  Less flexible for many-to-many relationships.	- Simple and efficient for hierarchical data. - Fast access. - Limited flexibility.
2. NETWORK MODEL 	- Data is organized as a graph with many-to-many relationships. - Records can have multiple parents.	 Telecommunication networks  Airline reservation systems  Supply chain management	 Efficient for complex many-to-many relationships.  Complex to design and maintain.	- Supports complex relationships. - Efficient data navigation. - More complex than hierarchical.
3. RELATIONAL MODEL 	- Data is represented in tables (relations) of rows and columns. - Relationships are established using keys.	 Banking systems  E-commerce platforms  University/College systems	 Highly flexible, easy to use and widely supported. Ideal for most business applications.	- Simple and intuitive. - Strong data integrity. - Uses SQL. - Scalable and robust.
4. ENTITY-RELATIONSHIP MODEL 	- Conceptual model using entities, attributes and relationships. - Used for designing databases.	 Database design  System analysis  Data modeling and documentation	 Best for conceptual design and communication. Not directly used for data storage.	- Easy to understand. - Good visualization. - Basis for converting to other models.
5. OBJECT-ORIENTED MODEL 	- Data is represented as objects with attributes and methods. - Supports inheritance, encapsulation, etc.	 CAD/CAM systems  Multimedia applications  AI and Simulation systems	 Suitable for complex data like images, videos, CAD, etc.  Less standardization compared to relational.	- Supports complex data types. - Inheritance and polymorphism. - Good for object-oriented applications.



Summary : Choose the data model based on the nature of data, relationships, performance needs, and application requirements.

SCHEMAS AND INSTANCES IN DBMS

Relationship between Physical and Logical Data Independence

KEY CONCEPTS

- Schema:** The blueprint or structure of the database at a particular level.
- Instance:** The actual data stored in the database at a particular time.

THREE LEVEL ARCHITECTURE

EXTERNAL LEVEL (View Level)

Multiple user views of the database.



CONCEPTUAL LEVEL (Conceptual Schema)

Logical structure of the entire database.



INTERNAL LEVEL (Internal Schema)

Physical storage structure of data.



SCHEMAS AND INSTANCES

SCHEMA (Structure)

EXTERNAL SCHEMA (View Schema)

Defines how a user sees the data.

INSTANCE (Data)

EXTERNAL INSTANCE (View Instance)

Actual data as seen by a user.

Roll No	Name	Course
101	Ramesh	DBMS
102	Sita	DBMS
...	...	...

CONCEPTUAL SCHEMA (Conceptual Level)

Defines the logical structure of the whole database.



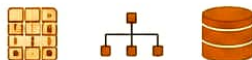
CONCEPTUAL INSTANCE (Conceptual Level)

Actual data in the logical structure.

Roll No	Name	Course
101	Ramesh	DBMS
102	Sita	DBMS
103	John	OS
...	...	...

INTERNAL SCHEMA (Internal Level)

Defines how data is stored physically.



INTERNAL INSTANCE (Internal Level)

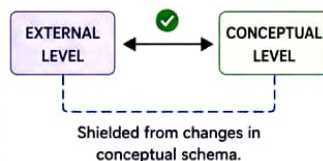
Actual data in the physical storage.

```
1010101010101010
0101001101010101
1010101110101001
...
```

DATA INDEPENDENCE

1. LOGICAL DATA INDEPENDENCE

Changes in the conceptual schema do not affect external schemas and user views.



Example:

Adding a new attribute in the conceptual schema does not affect user views.

2. PHYSICAL DATA INDEPENDENCE

Changes in the internal schema do not affect the conceptual schema.



Example:

Changing file organization or adding an index does not affect the logical structure.



SUMMARY

Schemas describe the structure at different levels, while instances represent the actual data at that time.

Data independence ensures changes at one level do not impact other levels, making the DBMS flexible and robust.



1. DATABASE ADMINISTRATOR (DBA)

Overall control of the database system and responsible for its proper functioning.

Responsibilities

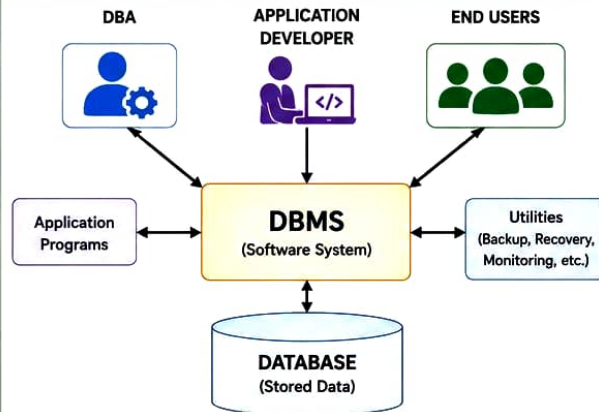
- ✓ Define database schema and storage structures.
- ✓ Create and manage user accounts and authorization.
- ✓ Ensure data security and integrity.
- ✓ Handle backup and recovery.
- ✓ Monitor performance and tune the system.
- ✓ Manage concurrency control and transactions.
- ✓ Ensure availability and reliability of the database.
- ✓ Handle hardware and software environment.

Example



DBA creates tables, sets permissions, performs backup, and ensures the system runs smoothly.

DBMS ENVIRONMENT



FLOW EXPLANATION

- Direct interaction
- ↔ Supports / Uses

All roles interact with the DBMS which manages the database and provides services to users.



2. END USER

Users who access the database through applications to perform various tasks.

Responsibilities

- ✓ Use applications to perform queries and transactions.
- ✓ Enter, view, update and delete data as per permissions.
- ✓ Generate reports and retrieve information.
- ✓ Follow data policies and security rules.
- ✓ Provide requirements and feedback for improvements.

Example



A student uses the application to view marks, a bank employee updates customer details, a salesperson generates reports.



3. APPLICATION DEVELOPER

Designs and develops applications that interact with the database.

Responsibilities

- ✓ Analyze user requirements and design application logic.
- ✓ Write programs and queries to access the database.
- ✓ Design user interfaces and reports.
- ✓ Test and debug applications.
- ✓ Ensure efficient interaction between application and DBMS.

Example



A developer creates a payroll system application that allows employees to update information and generate salary reports from the database.

KEY POINTS

- All roles are essential for the effective functioning of a DBMS.
- DBA ensures security, integrity and availability.
- Developers build applications to access data.
- End users use the data for their daily tasks.
- Together, they make the database system reliable, efficient and useful.



SUMMARY

DBA manages the system, Developers build the applications, and End Users use the data. The DBMS acts as an intermediary that ensures secure, efficient and reliable data management.

